Measurement Scope Matters: A Validation-First Comparison of AES-NI and GPU AES

Marko Mihajlovic[1]

Abstract: GPU kernel throughput alone does not establish whether AES offloading benefits a system with CPU AES-NI. We compare validated AES-128 ECB and CTR execution across three configurations, using paired inputs, five sessions per system, and separate resident and prepared-buffer pipeline measurements. At 32 MiB, resident GPU execution exceeds AES-NI in every session for both modes on all systems. Pipeline results differ: CTR achieves median paired speedups of 1.354× on T600 and 4.761× on T4, with gains in all five sessions, whereas GTX achieves 0.935× with only one session above parity. T4 ECB reaches 0.893× and loses in every session. The findings show that GPU offloading can outperform AES-NI, but its advantage depends on cipher mode, workload size, host performance, and measurement scope. The pipeline excludes preparation and application I/O; these results do not establish full application performance.

Keywords: Advanced Encryption Standard (AES), AES-NI, GPU acceleration, CUDA, heterogeneous computing, cryptographic benchmarking, data-transfer overhead, reproducible performance evaluation.

1 Introduction

AES acceleration is relevant to communication and data-processing systems that must encrypt large volumes without excessive latency. CPUs equipped with AES-NI provide a strong reference for this task, while GPUs offer parallel execution for independent blocks. The practical question is whether a GPU advantage survives the operations needed to move data through the accelerator. A fast kernel does not necessarily provide a faster host-to-host processing path.

Existing work demonstrates substantial GPU AES throughput and recognizes the importance of transfer costs and optimized CPU references. Yet a result measured on one implementation and timing boundary cannot determine whether offloading is beneficial on another system. Useful evidence must identify the CPU path, preserve workload pairing, distinguish device computation from transfer-inclusive processing, and report variability rather than a selected best time. This is particularly relevant to accessible laptop GPUs, whose suitability cannot be inferred from results obtained on a different host or accelerator.

We address this problem through a validation-first comparison of AES-NI

[1] Singidunum University, Belgrade, Serbia; ORCID: 0009-0006-7979-0341

and a validated CUDA AES-128 implementation. The study examines ECB and CTR across input sizes and three computing configurations. It addresses three research questions:

RQ1 Under which tested sizes and cipher modes does GPU execution outperform AES-NI, and which advantages are consistent across the observed sessions?

RQ2 How does including transfers change the magnitude and direction of the resident-only CPU–GPU comparison?

RQ3 How do session variability, mode dependence, workload-size effects, and differences among systems constrain the interpretation of GPU acceleration?

The paper makes three contributions: a reproducible protocol combining cryptographic validation, paired workloads and explicit timing scopes; a session-based assessment of AES-NI and GPU execution on the selected systems; and an interpretation that retains positive, negative and mixed-session outcomes. The contribution is a defensible comparison of measured execution paths, not a new AES primitive, a throughput record, or a universal hardware ranking.

Section 2 positions the study within earlier research. Section 3 defines the platforms, protocol and estimators. Section 4 reports the measurements, and Section 5 considers their practical meaning and limitations. Section 6 answers the research questions.

2 Related Work

Early GPU AES studies established the feasibility of exploiting block-level parallelism [1, 2]. Subsequent work examined thread granularity, memory placement and overlapping computation with transfers [3]. Lookup-table, shared-memory and bit-sliced implementations further demonstrate how representation and memory access affect GPU throughput [4–6]. These studies motivate GPU acceleration, but their rates are specific to the implementation, hardware and measured interval. Kernel optimization results cannot be treated as interchangeable with transfer-inclusive measurements.

The CPU reference is equally important. AES-NI accelerates AES transformations through dedicated instructions [7]. More generally, Lee et al. [8] show why CPU–GPU comparisons depend on the implementations selected as references. A gain over an unaccelerated CPU path therefore does not establish a gain over AES-NI. The present study uses the latter as its CPU denominator, while recognizing that a selected library path is not an upper bound on host performance.

Data location and movement are established concerns in heterogeneous benchmarking. Gregg and Hazelwood [9] emphasize their role in interpreting CPU–GPU results; AES-specific research has also considered transfer overlap [3]. CUDA and CuPy guidance additionally require suitable synchronization when timing asynchronous device operations [10, 11]. The distinction between resident execution and a broader processing path is therefore not introduced here as a novel concept.

Reliable reporting also requires a suitable replication structure. Mytkowicz et al. [12] demonstrate how experimental conditions can bias measurements. Kalibera and Jones [13] address variability across replication levels, while Hoefler and Belli [14] discuss defensible reporting in parallel computing. These principles motivate paired session estimates and explicit limits on inference.

The gap addressed here is an integrated, auditable assessment of when measured GPU advantages survive both an AES-NI reference and a transfer-inclusive boundary across the selected systems. The study combines established benchmarking principles with cryptographic validation and archived evidence, reporting consistent gains alongside reversals and variable outcomes. It complements implementation-optimization studies rather than attempting to rank their kernels or reproduce their performance records.

3 Methodology

3.1 Platforms and Implementations

Table 1 defines the system identifiers used throughout the paper. The operating-system editions and Colab identity are supplied labels; captured kernel and software versions are retained in the environment records. The exact cloud CPU SKU is not available. Each identifier denotes a complete hardware–software configuration, not an isolated GPU or operating-system effect.

The CPU path uses PyCryptodome with AES-NI explicitly enabled [15]. Native backend initialization is checked separately for ECB and CTR; this supports the intended dispatch but is not an instruction trace. The CUDA path is the validated implementation archived as `cuda_baseline`, invoked through CuPy [16]. It assigns one AES block to each thread, with 128 threads per CUDA block. This implementation is a reproducible comparator, not a claim of maximum GPU AES performance.

The article selects this CUDA path and AES-NI from a broader archived campaign. Other CPU and CUDA paths were interleaved in the original execution schedule but are excluded from the reported comparisons. No measurements were rerun or altered for this selection. It retains 9,000 observations from the original

Table 1
Experimental configurations and software stacks.

ID	Hardware and environment	Software stack
GTX	Intel Core i5-10300H; GeForce GTX 1650 Ti, 4 GB; Windows 10	Driver 610.62; CUDA runtime 12.9; CuPy 13.6.0; PyCryptodome 3.23.0
T600	Intel Core i7-11850H; T600 Laptop GPU, 4 GB; Windows 11	Driver 581.95; CUDA runtime 12.9; CuPy 14.2.0; PyCryptodome 3.24.1
T4	Intel Xeon at 2.00 GHz, two exposed logical processors; Tesla T4; Linux/Google Colab	Driver 580.82.07; CUDA runtime 13.0; CuPy 14.0.1; PyCryptodome 4.0.0

18,000, while preserving all fifteen sessions and every size/mode combination for the selected paths.

3.2 Validation and Paired Workloads

Cryptographic checks precede accepted performance measurements. Known-answer tests and cross-backend comparisons use the AES and mode specifications [17, 18]. The archived validation records document successful CUDA execution and agreement with CPU references. Correctness is necessary but does not establish timing reliability; completeness, configuration consistency and artifact integrity are checked separately.

The workload is AES-128 encryption in ECB and CTR at 1 KiB, 16 KiB, 1 MiB, 16 MiB and 32 MiB. ECB is a performance probe, not a recommendation for securing structured application data. Inputs and cryptographic parameters are generated deterministically and shared by the paired CPU and GPU jobs. Each system contributes five sessions, with seeds 2003–2007, five warmups and twenty measured repetitions per group. Warmups are excluded from the exports and are supported by configuration and source provenance. The retained subset has thirty groups and 600 observations per session.

Sizes execute in fixed ascending order; mode and backend/scope jobs are shuffled using the session seed. Experiments run sequentially, without concurrent benchmark jobs. The original ordering is retained in the evidence, including jobs excluded from this article. Sessions are repeated executions on the same host, not independently sampled machines.

3.3 Measurement Boundaries

Host wall timing uses `time.perf_counter_ns()`. The three intervals are distinct:

- **CPU transform:** the encryption call, including output allocation but excluding cipher construction and key setup.
- **GPU resident:** a single kernel launch timed with CUDA events; transfers occur outside the interval. The resident-iteration count is one.
- **GPU pipeline:** host wall time for the synchronous H2D–kernel–D2H sequence, including launch and required synchronization, using prepared pinned host buffers and preallocated device buffers.

GPU compilation, allocation, pinning, initial staging, key preparation and application I/O are outside these intervals. The pipeline is consequently not full application encryption. Resident event timing and pipeline wall timing are not interchangeable observations of one interval [11]. Recorded copy intervals include synchronization; subtracting resident time from pipeline time cannot identify pure PCIe transfer latency.

3.4 Statistics and Reproducibility

For payload size N bytes and elapsed time $T_{s,r}$ in session s, repetition r, throughput is $R_{s,r} = N/(10^6 T_{s,r})$ in decimal MB/s. Input labels use binary KiB and MiB. We first compute the median of each session's twenty latency and throughput observations, then aggregate across sessions:

$$\widetilde{T} = \mathrm{median}_s\big(\mathrm{median}_r\, T_{s,r}\big), \qquad \widetilde{R} = \mathrm{median}_s\big(\mathrm{median}_r\, R_{s,r}\big). \tag{1}$$

Throughput is summarized independently rather than obtained by inverting the aggregate latency. For each size, mode and GPU scope, paired speedup is

$$S_s = \frac{\widetilde{T}_{\text{AES-NI},s}}{\widetilde{T}_{\text{GPU},s}}, \qquad \widetilde{S} = \mathrm{median}_{s=1,\ldots,5}\, S_s. \tag{2}$$

Values above one favor the GPU. We report session ranges, interquartile ranges and sample coefficients of variation (CV = sample standard deviation/mean), together with directional counts. A $5/5$ gain is observed consistency, not a significance test. Repetitions are not pooled as independent sessions, no outliers are removed, and systems retain their own CPU references.

Configurations, raw observations, validation records, source snapshots and SHA-256 hashes are archived under one selected run per system. The retained estimators were independently reconstructed from native raw CSVs, including the authoritative T4 archive, and agreed with saved aggregates within relative tolerance 10^{-10} and absolute tolerance 10^{-15}. Active benchmark source snapshots match across systems; software environments and compiled binaries need not. The

accompanying evidence mapping records run IDs, seeds, raw-file hashes and the selection, while the manuscript's offline figure script reproduces the retained tables and plot without executing benchmarks.

4 Results

4.1 Offloading Advantage and Timing Scope

Figure 1 compares resident and pipeline speedups across the complete size grid. The numerical contrast at 32 MiB is summarized in Table 2. Each comparison uses the corresponding system's AES-NI reference and the estimators defined in Section 3.4.

Table 2
Paired speedup over AES-NI at 32 MiB. R: resident; P: pipeline. Brackets show the five-session pipeline range; the last column counts P gains.

System	Mode	R	P	P range	P > 1
GTX	ECB	3.091	0.761	[0.722, 0.818]	0/5
GTX	CTR	3.693	0.935	[0.918, 1.008]	1/5
T600	ECB	2.215	1.118	[0.659, 1.144]	3/5
T600	CTR	2.666	1.354	[1.129, 1.391]	5/5
T4	ECB	2.207	0.893	[0.868, 0.901]	0/5
T4	CTR	11.758	4.761	[4.691, 5.021]	5/5

Resident execution exceeds AES-NI in all five sessions for both modes on every system at 32 MiB. Transfer-inclusive results differ. GTX CTR reaches 0.935×, with a range of 0.918–1.008× and only one session above parity; its ECB pipeline loses in all five sessions. No tested GTX pipeline condition has an all-five-session advantage over AES-NI.

T600 CTR provides the clearest consistent laptop result: a median pipeline speedup of 1.354× at 32 MiB, with gains in all five sessions. This corresponds to approximately 26.14% lower paired latency. The speedup IQR is 0.030896 and CV is 8.18%. CTR also wins in all five sessions at 1 MiB, whereas the direction is mixed at 16 MiB. T600 ECB at 32 MiB has a median gain but loses in two sessions.

T4 CTR reaches 4.761× at 32 MiB and wins in all five sessions. Its ECB pipeline reaches only 0.893× and loses in every session. Thus even a consistent resident advantage does not establish a corresponding pipeline advantage.

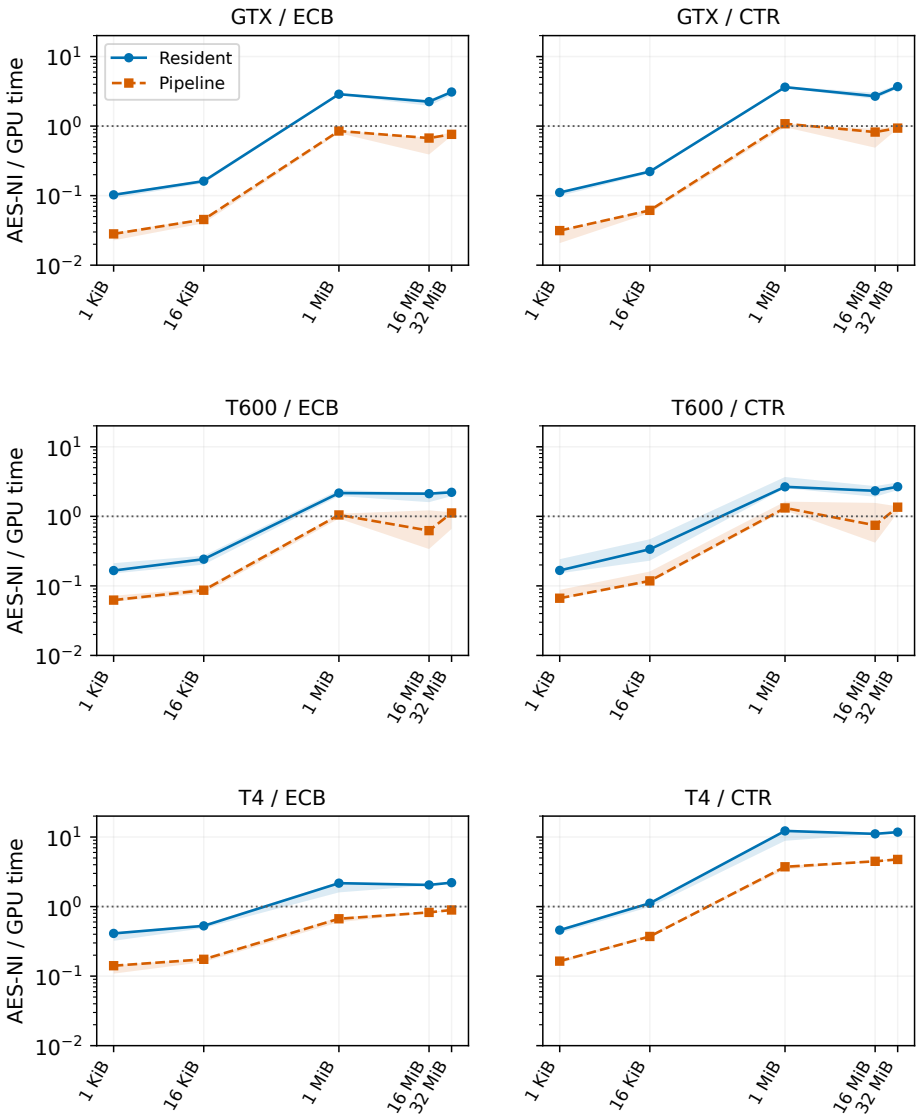


Fig. 1 — AES-NI versus GPU execution by size and mode. Lines show median paired speedups; shading spans the five session estimates. The dotted line denotes parity. Resident and prepared-buffer pipeline are distinct scopes. Ranges are descriptive, not confidence intervals. Input sizes use binary units.

4.2 Absolute Throughput and Cipher Mode

At 32 MiB CTR, pipeline throughput is 1188.9 MB/s on GTX, 2049.4 MB/s on T600 and 3713.9 MB/s on T4. Resident throughput is 4585.6, 4328.9 and 9166.8 MB/s, respectively. These absolute GPU rates are distinct from the CPU-relative speedups in Table 2.

T4 pipeline throughput is nearly equal for ECB and CTR at this size, approximately 3713 and 3714 MB/s. Its host AES-NI ECB throughput, however, is 5.378× the CTR throughput, with session ratios ranging from 5.250 to 5.795×. The corresponding ratios on GTX and T600 are approximately 1.26×. The T4 disparity is consistent across sessions, but its cause was not isolated. A large paired speedup can therefore reflect CPU mode behavior as well as GPU throughput; it is not a direct measure of the GPU's advantage over another system.

4.3 Size Dependence and Variability

Throughput is not uniformly monotonic in input size. All four retained GTX mode/scope groups show a local 16 MiB dip below both 1 and 32 MiB in all five sessions. T4 resident measurements show that dip in four sessions, whereas its pipeline measurements show none. The size feature therefore depends on the configuration and measurement scope.

Session ranges also matter when the median is near parity. The mixed T600 ECB outcome and occasional GTX CTR gain at 32 MiB would be obscured by reporting only their medians. Together with the size-dependent reversals, these observations support retaining the full workload grid and session-level comparisons rather than selecting a single favorable measurement.

5 Discussion

5.1 Practical Meaning of Offloading

The results answer the offloading question conditionally. T600 and T4 demonstrate that a prepared-buffer GPU pipeline can outperform AES-NI for a measured CTR workload. GTX and T4 ECB show why neither GPU availability nor high resident throughput is sufficient to justify that choice. For data already on the device, resident execution is relevant; for host-resident input and output, the pipeline comparison is more informative. Neither substitutes for measuring the application's complete processing path.

The T4 mode contrast also shows why CPU-relative speedup and absolute GPU throughput must be reported separately. Similar GPU rates can produce very different speedups when host CPU modes behave differently. The observations do not identify a particular library version, CPU feature or cloud mechanism as the

cause. A controlled same-host investigation would be needed to separate these possibilities.

For telecommunications, the positive large-buffer results motivate evaluation of bulk or batched processing, not a claim that individual packets are encrypted faster on a GPU. Packet assembly, queueing, state management, authentication and output delivery can change the trade-off. AES-NI remains a relevant reference for latency-sensitive workloads; a system should select an execution path using its actual data location and processing requirements.

5.2 Limitations and Future Work

The study evaluates one host or cloud allocation per configuration and five sessions per host. It describes repeatability within these runs, not population-wide reliability or statistical significance. Hardware, operating systems, drivers and libraries differ simultaneously, so the comparison cannot isolate an operating-system or GPU-class effect. Matching archived source bytes do not establish identical compiled code or execution conditions.

The CPU reference is a selected library path, not the fastest possible multi-threaded AES service, and the CUDA implementation is not a GPU performance limit. The reported subset was selected from a broader interleaved campaign; removing other paths from the article does not remove their possible effects on the original execution state. Source provenance and integrity checks support auditability but cannot eliminate timing bias or environmental interference.

The fixed ascending size order confounds size with elapsed execution time. The 16 MiB feature cannot be attributed to cache, driver, clock or thermal behavior from the available evidence. Randomized, more densely sampled sizes with synchronized instrumentation would help distinguish these hypotheses. Additional hosts and repeated cloud allocations would improve external validity, while controlled CPU-mode measurements would clarify the T4 disparity.

Finally, ECB and CTR are not authenticated encryption. AES-GCM adds GHASH and associated security processing [19]; its performance and that of complete network protocols do not follow from these results. Application evalu-

ations must include authentication, preparation and delivery costs.

6 Conclusion

For RQ1, the measured GPU pipeline provides consistent CTR gains over AES-NI on T600 and T4 at 32 MiB, but not on GTX; T4 ECB consistently favors the CPU. For RQ2, resident gains on all three systems do not survive transfer inclusion uniformly. For RQ3, mode-dependent CPU performance, non-monotonic size behavior and session variation limit generalization from a single speedup value.

The study's contribution is a validated, reproducible comparison that preserves these conditions and negative results. GPU offloading is a workload-dependent option rather than a universal replacement for AES-NI. The decision must be evaluated against the actual host CPU, data location and application boundary.

References

[1] Svetlin A. Manavski. Cuda compatible gpu as an efficient hardware accelerator for aes cryptography. In *Proceedings of the IEEE International Conference on Signal Processing and Communications*, 2007. doi: 10.1109/ICSPC.2007.4728256.

[2] Owen Harrison and John Waldron. Aes encryption and decryption on the gpu. In *GPU Gems 3*, chapter 36. Addison-Wesley Professional, 2007. URL `https://developer.nvidia.com/gpugems/gpugems3/part-vi-gpu-computing/chapter-36-aes-encryption-and-decryption-gpu`.

[3] Keisuke Iwai, Naoki Nishikawa, and Takakazu Kurokawa. Acceleration of aes encryption on cuda gpu. *International Journal of Networking and Computing*, 2(1):131–145, 2012. doi: 10.15803/ijnc.2.1_131.

[4] Canhui Wang and Xiaowen Chu. Gpu accelerated aes algorithm, 2019. URL `https://arxiv.org/abs/1902.05234`.

[5] Cihangir Tezcan. Optimization of advanced encryption standard on graphics processing units. *IEEE Access*, 9:67315–67326, 2021. doi: 10.1109/ACCESS.2021.3077551.

[6] Wai-Kong Lee, Seog Chung Seo, Hwajeong Seo, Dong Cheon Kim, and Seong Oun Hwang. Speed record of aes-ctr and aes-ecb bit-sliced implementation on gpus. *IEEE Embedded Systems Letters*, 16(4):481–484, 2024. doi: 10.1109/LES.2024.3409725.

[7] Intel Corporation. Intel advanced encryption standard instructions (aes-ni), 2012. URL `https://www.intel.com/content/www/us/en/developer/articles/technical/advanced-encryption-standard-instructions-aes-ni.html`.

[8] Victor W. Lee, Changkyu Kim, Jatin Chhugani, Michael Deisher, Daehyun Kim, Anthony D. Nguyen, Nadathur Satish, Mikhail Smelyanskiy, Srinivas Chennupaty, Per Hammarlund, Ronak Singhal, and Pradeep Dubey. Debunking the 100x gpu vs. cpu myth: An evaluation of throughput computing on cpu and gpu. In *Proceedings of the 37th Annual International Symposium on Computer Architecture*, pages 451–460, 2010. doi: 10.1145/1815961.1816021.

[9] Chris Gregg and Kim Hazelwood. Where is the data? why you cannot debate cpu vs. gpu performance without the answer. In *2011 IEEE International Symposium on Performance Analysis of Systems and Software (ISPASS)*, pages 134–144, 2011. doi: 10.1109/ISPASS.2011.5762730.

[10] NVIDIA Corporation. Cuda c++ best practices guide, n.d. URL `https://docs.nvidia.com/cuda/cuda-c-best-practices-guide/`. Accessed 2026.

[11] CuPy. Performance best practices. `https://docs.cupy.dev/en/stable/user_guide/performance.html`, n.d.. Accessed 2026.

[12] Todd Mytkowicz, Amer Diwan, Matthias Hauswirth, and Peter F. Sweeney. Producing wrong data without doing anything obviously wrong! In *Proceedings of the 14th International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 265–276, 2009. doi: 10.1145/1508244.1508275.

[13] Tomas Kalibera and Richard E. Jones. Rigorous benchmarking in reasonable time. In *Proceedings of the International Symposium on Memory Management*, 2013. doi: 10.1145/2464157.2464160.

[14] Torsten Hoefler and Roberto Belli. Scientific benchmarking of parallel computing systems: Twelve ways to tell the masses when reporting performance results. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*, pages 73:1–73:12, 2015. doi: 10.1145/2807591.2807644.

[15] PyCryptodome. Pycryptodome documentation, n.d. URL `https://pycryptodome.readthedocs.io/`. Accessed 2026.

[16] CuPy. cupy.rawkernel. `https://docs.cupy.dev/en/stable/reference/generated/cupy.RawKernel.html`, n.d.. Accessed 2026.

[17] National Institute of Standards and Technology. Advanced encryption standard (aes). Federal Information Processing Standards Publication 197, Update 1, National Institute of Standards and Technology, 2023.

[18] Morris Dworkin. Recommendation for block cipher modes of operation: Methods and techniques. NIST Special Publication 800-38A, National Institute of Standards and Technology, 2001.

[19] Morris Dworkin. Recommendation for block cipher modes of operation: Galois/counter mode (gcm) and gmac. NIST Special Publication 800-38D, National Institute of Standards and Technology, 2007.